



第三章 编程逻辑

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院



目录

- 1. 程序流程图**
- 2. 顺序结构**
- 3. 分支结构**
- 4. 循环结构**
- 5. 应用案例**
- 6. 本章习题**



1. 程序流程图

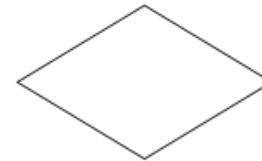
- **编程逻辑**：一是顺序执行；二是条件执行；三是循环执行。
- **程序流程图**



(a) 起止框



(b) 处理框



(c) 判断框



(d) 数据框



(e) 子程序框

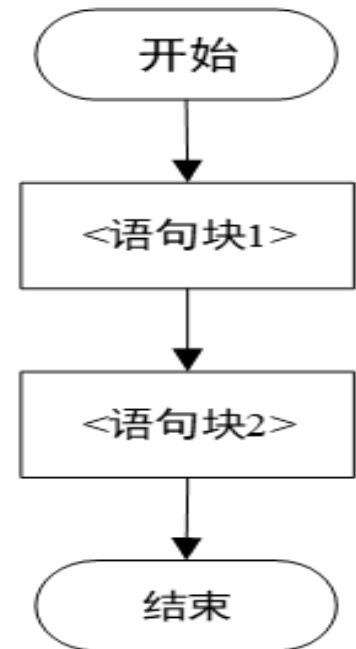


(f) 流向线



2. 顺序结构

- 程序的**顺序结构**：指程序从上至下按行依次按行执行。
- 语法结构：
 - <语句块1>
 - ...
 - <语句块N>
- 在<顺序结构的语句块>中，其中的变量的作用域是自其声明之后的代码段



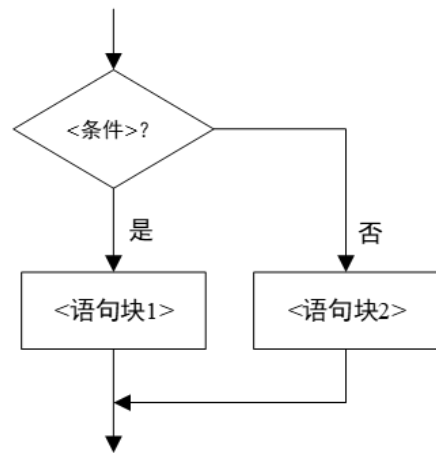
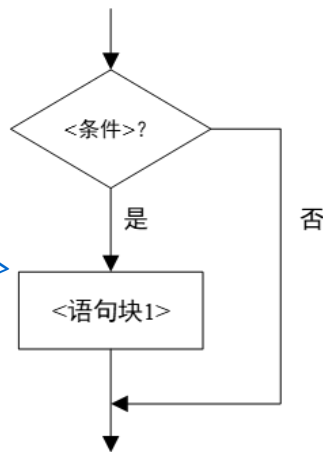


3. 分支结构

- **分支结构**：程序有一个或多个条件分支，满足某一条件，则执行该条件对应的分支，其他不满足条件的分支，不予执行。

- 语法结构：

- **if** <条件1>: # 若<条件1>满足, 则执行<语句块1>
 <语句块1>
 ...
- **elif** <条件i>: # 否则, 但满足<条件i>, 则执行 <语句块i>
 <语句块i>
 ...
- **else**: # 若以上所有条件均不满足, 则执行<语句块n>
 <语句块n>



- 形成判断条件最常用的方式是采用关系操作符，包括<、<=、>、>=、== 和 !=。
- 在实际编程中，分支结构中的条件个数可以是1个或多个。
- **注意**：符号=表示赋值，符号==表示判断符号前后两个操作数是否相等



简单示例

- 示例:

```
if age > 18:
```

```
    print("你已成年")
```

```
elif age > 12:
```

```
    print("你是青少年")
```

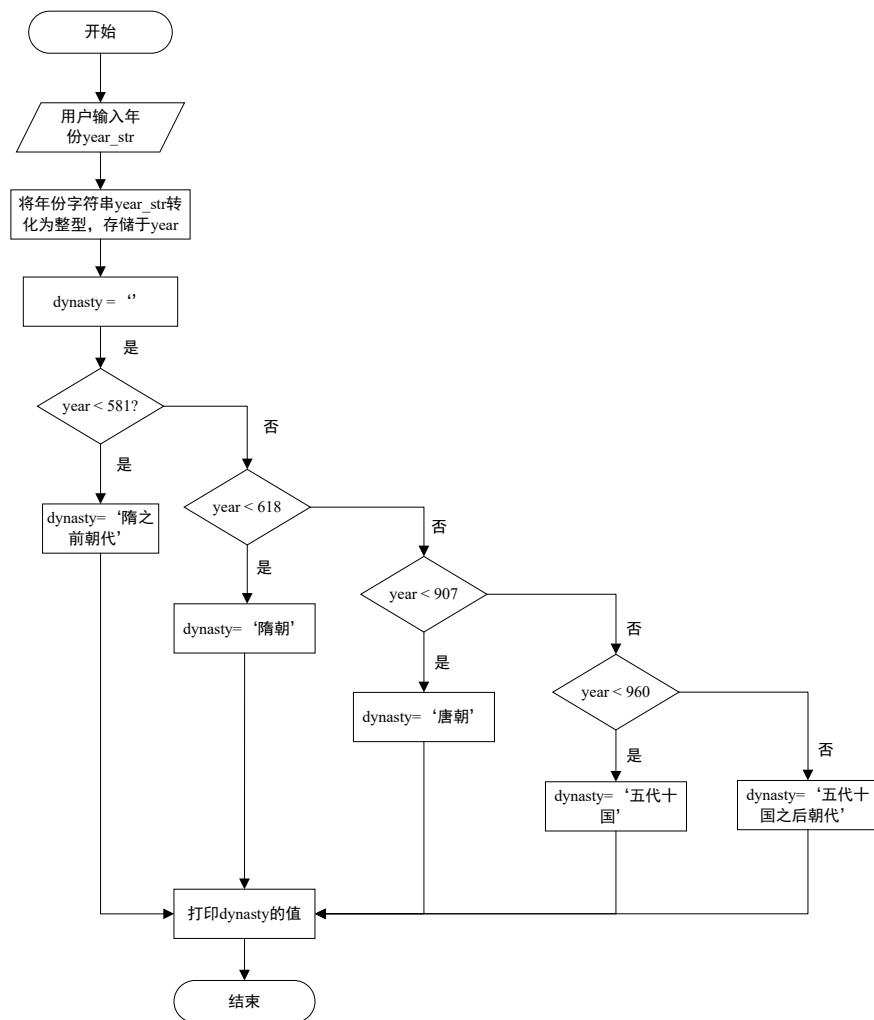
```
else: # <=12
```

```
    print("你是儿童")
```



举例：判别中国历史朝代

- 在中国历史中，隋、唐、五代十国的年代表如下：
 - 隋：581-618
 - 唐：618-907
 - 五代十国：907-960
- 给定年份判断其所处朝代的代码，主要采用分支结构





- 历史朝代判别算法代码如下：

```
year_str = input("请输入历史年份：")
year = int(year_str)
dynasty = ""
if year < 581:
    dynasty = '隋之前朝代'
elif year < 618:
    dynasty = '隋朝'
elif year < 907:
    dynasty = '唐朝'
elif year < 960:
    dynasty = '五代十国'
else:
    dynasty = '五代十国之后朝代'
print('公元{0}年是{1}'.format(year, dynasty))
```




模式匹配

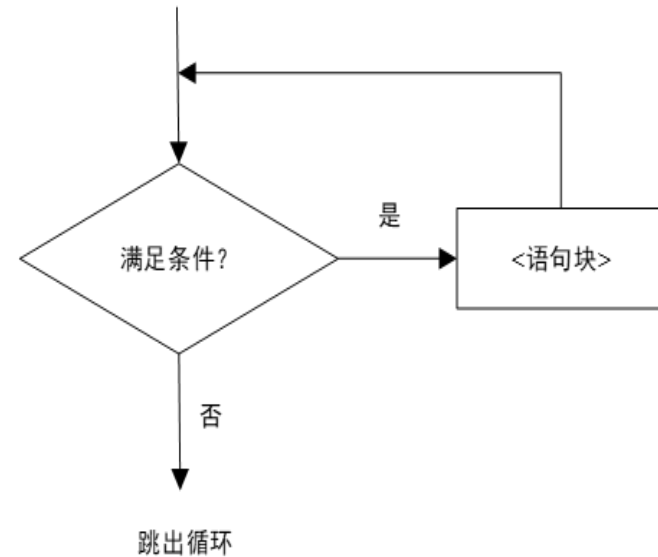
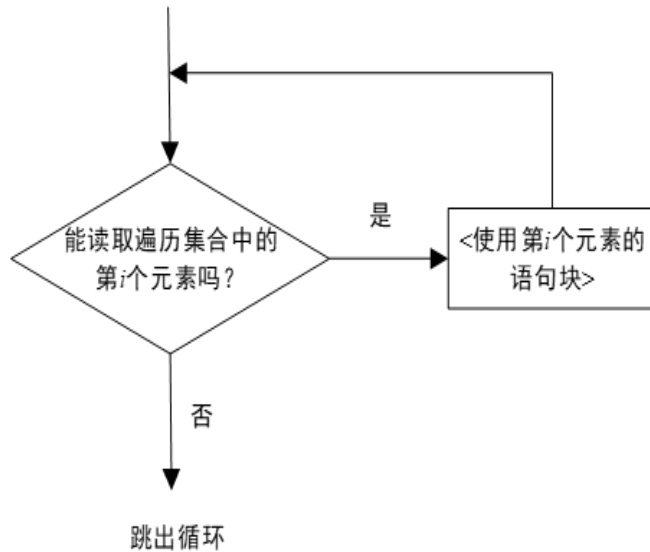
- Python 3.10 引入了**模式匹配**的新功能：这是一种类似于其他编程语言中 switch-case 语句的结构，但功能更强大和灵活。在 Python 中，这个特性通过 match 和 case 关键字实现，它根据对象的结构和内容来进行分支处理。
- 使用模式匹配的示例：

```
◦ def handle_message(message):  
    match message:  
        case {"type": "text", "content": text}:  
            print(f"Text message: {text}")  
        case {"type": "image", "content": image_path}:  
            print(f"Image message with path: {image_path}")  
        case {"type": "video", "duration": duration}:  
            print(f"Video message of duration {duration} seconds")  
        case _:  
            print("Unknown message type")  
message1 = {"type": "text", "content": "Hello World"}  
message2 = {"type": "image", "content": "/path/to/image.jpg"}  
message3 = {"type": "video", "duration": 120}  
handle_message(message1)  
handle_message(message2)  
handle_message(message3)
```



4. 循环结构

- **循环结构**：当程序在只有个别变量变化的情况下，需要反复执行相似逻辑，直到满足某一个条件后，退出循环。一般有两种实现方法：
 - **for循环**：遍历循环
 - **while循环**：条件循环





for循环

- **for循环**：是一种遍历循环，即遍历一个序列中的每一项，用于迭代一个序列中的每一项，并执行相应代码
 - 该序列可以是**列表**、**元组**、**字符串**或**生成器等可迭代对象**
- **for循环语法结构**：
 - **for** <循环变量> **in** <遍历集合>:
 <语句块1>
 else: # 通常可省略
 <语句块2>
- 举例：
 - **for** i **in** range(1, 5):
 print(i)
 - 函数说明：**range(start, stop[, step])**，其中参数
 - start: 计数从 start 开始。默认从 0 开始，如range(5)等价于range(0, 5);
 - stop: 计数到 stop 结束，**但不包括 stop**，如上例。
 - step: 步长，默认为1，如：range(0, 5)等价于 range(0, 5, 1)
- **思考**：如何将所有的数字打印在一行？



while循环

- for 循环在预知循环次数时使用，而 **while 循环** 则用于在不知道具体循环次数时的条件循环。只要满足 while 语句的条件，循环将持续执行，直到条件不成立为止
- **while 循环语法结构**
 - **while** <条件>: # <条件> 为布尔表达式，结果为 True 或 False
 <语句块1>
 - else:** # 本分支和 for 循环的 else 类似，通常可省略
 <语句块2>
 - while 循环中，当 <条件> 为 True 时，循环执行 <语句块1>，当 <条件> 为 False 时，循环终止。当 while 循环正常执行后，程序会继续执行 else 语句中的 <语句块2>，否则，不会执行 else 语句。<条件> 的变化是在 <语句块> 中被设置的。
- **思考：**如何使用循环语句来判断用户输入的正整数是否为素数？素数的定义为“在大于1的自然数中，除了1和该数自身外，无法被其他自然数整除的数”。



举例：猜数字游戏

- `number = 62` *# 程序中预设一个答案, 供用户猜*

`correct = False`

while not correct:

`guess = int(input('请输入一个从0到100范围内整数: '))`

if guess == number:

`print('恭喜你, 你猜对了! ')`

`correct = True;`

elif guess < number:

`print('猜错了, 你猜的数太小了')`

elif guess > number:

`print('猜错了, 你猜的数太大了')`

`print('程序结束')`

- **思考：**如何添加关于错误大小的信息提示？比如，用户猜测与正确答案差距的绝对值小于等于5时，提示“你猜的数和预设的数相差不远了（差距小于等于5）”。



break语句

- 在程序执行时，通常情况下，只有当 for 语句中的遍历条件或 while 语句中的循环条件不满足时，程序才会跳出循环。然而，在某些情况下，可能需要**在满足特定条件时立即退出循环**。为此，大多数编程语言中都引入了 **break 语句**来实现该功能
- break 语句**：通常出现在 for 或 while 循环的语句块中，通常与 if 条件语句一起使用。它的作用是强制跳出循环，无论循环条件是否满足，或者迭代次数是否达到预设值，一旦执行到 break 语句，程序将立即跳出当前循环
- Break语句和语法**

- **while** <条件1>:

...

if <条件2>:

break

...

- **for** <循环变量> **in** <遍历集合>:

...

if <条件2>:

break

...

- **举例**：将所有小写的英文字母，转化为大写

while True:

str = input("请输入一个英文字符串： ")

if str == 'exit' : # 当用户输入 'exit' 字样, 则跳出循环

break

print("大写： " , str.upper()) #upper()将小写字母变为大写字母

print("程序结束")



continue语句

- break 语句用于强制跳出整个循环。然而，有时并不需要完全退出循环，而是希望中断当前迭代，跳过本次循环的剩余部分，直接进入下一次迭代。为此，大多数编程语言引入了 **continue 语句**
- **continue 语句**：跳过当前迭代中尚未执行的代码，立即进入下一次迭代。它的使用位置与 break 语句类似，通常用于 if 条件语句中
- **continue 语法结构**

- while <条件1>:

...

if <条件2>:

continue

...

- for <循环变量> in <遍历集合>:

...

if <条件2>:

continue

...

- 举例：将所有小写的英文字母，转化为大写

while True:

 str = input("请输入一个英文字符串： ")

if str == 'exit' :

break

if not str.isascii():

 print("输入非英文字符串")

continue

 print("大写：", str.upper())

print("程序结束")



海象操作符

- **海象操作符 (Walrus Operator)**：是一种新的赋值表达式，表示为 `:=`。它允许在表达式中进行赋值并返回赋值结果，从而使代码更加简洁和高效。
- **海象操作符的主要用途**：在条件表达式或循环中**同时进行变量赋值和条件检测**，减少代码量并提高可读性

1. 在 while 循环中使用

海象操作符可以在 while 循环中使用，以减少重复代码。

不使用海象操作符

```
n = get_value()
while n > 0:
    process(n)
    n = get_value()
```

使用海象操作符

```
while (n := get_value()) > 0:
    process(n)
```

2. 思考：如何使用海象操作符改造案例“猜数字游戏”，会带来问题么？



- 在 if 语句中使用

- 海象操作符还可以在 if 语句中使用, 同时进行赋值和条件判断

- # 使用海象操作符

```
if (result := calculate_value()) > 10:  
    print(f"Result is large: {result}")
```

- # 不使用海象操作符

```
result = calculate_value()  
if result > 10:  
    print(f"Result is large: {result}")
```



5. 案例1：斐波那契数列

- **斐波那契数列**：数学家莱昂纳多·斐波那契研究兔子繁殖问题时，提出了斐波那契数列（Fibonacci sequence），又称“兔子数列”。生成斐波那契数列的方法如下：
 - $F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2) (n \geq 2, n \in \mathbb{N})$
 - 随着该数列项数的增加，前一项与后一项之比越来越逼近黄金分割数值0.618，因此该数列又称为“黄金分割数列”
 - 利用Python编程生成100之内的斐波那契数列，并验证其黄金分割现象

```
x1, x2 = 0, 1
```

```
max_number = 100
```

```
while x2 < max_number:
```

```
    ratio = round(x1/x2, 5) if x2 != 0 else 0 # 避免除以0的情况；这个判断冗
```

```
    余么？
```

```
    print(x2, ratio)
```

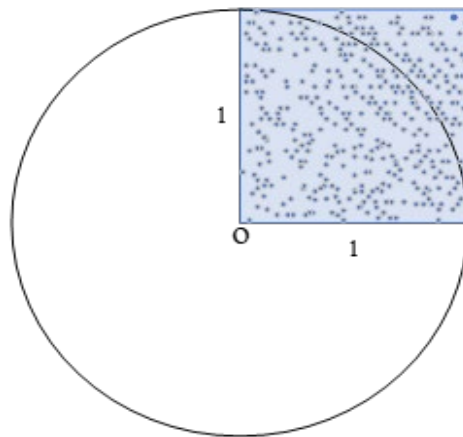
```
    x1, x2 = x2, x1 + x2
```

- **与AI协作探索**：三元表达式“变量 = 表达式1 if 条件表达式 else 表达式2”的含义及用法



案例2：计算圆周率

- **蒙特卡罗方法：**一般认为美国在第二次世界大战中研制原子弹的“曼哈顿计划”中首先提出。但实际上，1777年，法国数学家布丰就提出用投针实验的方法求圆周率 π
- **使用蒙特卡罗方法求解 π ：**
 - 边长为1的单位正方形中占据半径为1的单位圆的右上方，向单元正方形范围内，随机投掷 n 个飞镖后，统计落在了右上方的1/4圆内的飞镖数量为 m 。根据统计模拟方法，1/4圆的面积（即 $\pi/4$ ）与单位正方形面积（即1）之比，约等于落在1/4圆内飞镖数量 m 与落在单位正方形内的飞镖数 n 之比，即 $\pi/4 = m/n$ ，也即 $\pi = 4m/n$





- **实现代码：**

```
from random import random
from math import sqrt
n = 10000
count_in_circle = 0
for i in range(0, n):
    x, y = random(), random()
    dist = sqrt(x**2 + y**2)
    if dist <= 1:
        count_in_circle += 1
pi = 4 * count_in_circle / n
print("Pi's value is", pi)
```

- **与AI协作探索：**蒙特卡罗方法在经济管理领域有哪些应用？如何使用Python编写一个应用案例？



案例3：广播模型

- 假设一个相关人群，人数为 N ，这里相关人群是指感知某一信息或感染某一疾病的潜在人群。在某一时刻 t ，人群可被划分为两类：
 - 一类是知情者或感染者 (Infective)，感染者数量用 I_t 表示
 - 另一类是还未感染的潜在人群，称为易感者 (Susceptible)，易感者数量用 S_t 表示。
 - 这里 $N = I_t + S_t$
- **广播模型：** $I_{t+1} = I_t + P_{broad}S_t$
 - 其中 I_t 和 I_{t+1} 分别表示第 t 和 $t + 1$ 时刻的感染者人数， P_{broad} 为感染概率， S_t 为第 t 时刻易感者人数
 - 广播模型刻画了新闻、信息和技术通过电视、广播、互联网等进行的传播，是一种一对多的传播方式



- **问题1：** 一个100个居民的小村庄，村长利用广播每天播放一次修路倡议，假设一个人在任何一天听到这个倡议广播的概率为10%（即 $P_{\text{broadcast}}=0.1$ ），问：多少天后全村人都从广播中听到了这个倡议

```
Number = 100
day = 0
infective = 0.0
infective_number = 0
probability = 0.1
infective_sequence_str = ""
while infective_number < Number:
    day += 1
    susceptible = Number - infective
    infective = infective + probability * susceptible
    infective_number = int(infective + 0.5)
    infective_sequence_str = infective_sequence_str + "{} ".format(infective_number)
print(infective_sequence_str)
print("In the {}th day, everyone receives the message from the broadcast.".format(day))
```



扩散模型

- 广播模型考虑了从一个原点向外部多个对象广播信息或病毒，在实际中，还存在着另一种信息传播的方式——**扩散模型**，即获得信息的多个感染者，可将信息或病毒传递给它们周围的对象，形成多对多的扩散传播方式
- 扩散模型**: $I_{t+1} = I_t + P_{diffuse} \cdot \frac{I_t}{N} \cdot S_t$
 - 其中 $\frac{I_t}{N}$ 为随机接触到感染者的概率， $P_{diffuse}$ 为扩散概率（接触后被传染的概率）
- 思考**: 假设每位易感者每一期有 $\tilde{M}$ 次与其他人随机接触的机会， $\tilde{M}$ 为1至5之间的随机整数（ $1 \leq \tilde{M} \leq 5$ ），且每次接触到的人为感染者的概率为 $\frac{I_t}{N-1}$ ，接触到感染者后被感染的概率为 $P_{diffuse}$ ，请使用**随机采样**的方法，模拟每次随机接触的结果，统计并打印感染者数量的演化路径。



- **问题2:** 一个100个居民的小村庄, 村长将修路倡议扩散给他周围的人, 该倡议在人们之间的扩散概率为10% ($P_{diffuse}=0.1$), 问: 多少天后全村人都知道了这个倡议

```
Number = 100
day = 1
infective = 1
infective_number = 1
probability_diffuse = 0.1
infective_sequence_str = str(infective_number) + ' '
while infective_number < Number:
    day += 1
    susceptible = Number - infective
    infective = infective + probability_diffuse * (infective/Number) * susceptible
    infective_number = int(infective + 0.5)
    infective_sequence_str = infective_sequence_str + "{} ".format(infective_number)
print(infective_sequence_str)
print("In the {}th day, everyone knows the message from the message diffusion.".format(day))
```




中国高铁与程序逻辑

- 计算机算法的核心主要由**顺序结构**、**分支结构**和**循环结构**构成
- 这些基本程序逻辑的组合，就能支撑起复杂系统的运行
- 例如，**高铁调度系统**就是通过精确的逻辑与算法，实现列车间隔控制、路径分配与安全调度
 - 中国高铁实现跨越式发展，建成世界上规模最大、技术最先进的高速铁路网，形成“八纵八横”主骨架。高铁已成为“**中国速度**”的象征，彰显**自主创新与科技强国**的成就

图表 11：2022-2023 年中国高(快)铁规划建设示意图



资料来源：国家铁路局 前瞻产业研究院



@前瞻经济人APP



习题

• 理论知识题：

1. 解释顺序结构在程序设计中的作用，并给出一个简单的示例。
2. 阐述分支结构在决策制定中的重要性，并举例说明其在实际编程中的应用。
3. 比较for循环和while循环的不同用途，并解释在什么情况下应该选择使用何种循环。
4. 选择本章中的一个编程案例，绘制其流程图，并解释各个元素的含义。
5. 一个循环结构中是否还可以嵌套另一个循环结构？请举例说明。
6. 在运行带有循环结构的程序时，是否发现程序死机现象？这可能是由于什么导致的？



习题

• 编程实践习题

1. 编写一个程序，使用嵌套循环打印5x5的星号矩阵。
2. 编写程序实现根据输入的成绩（0-100）来判断并输出成绩的等级，85分以上为优、75分到84分为良、60分到74分为及格、60分之下为不及格。
3. 编写一个程序，输入一个年份，判断该年份是否为闰年（提示：闰年条件为年份能被4整除且不能被100整除，或者能被400整除）
4. 使用循环编写程序，计算1到100的所有整数之和。
5. 使用循环编写程序，计算前n个偶数的和，其中n由用户输入。
6. 编写程序输出所有1到100之间的所有素数。
7. 编程解决百钱百鸡问题，即：公鸡5钱一只，母鸡3钱一只，小鸡3只1钱。现在用100钱买100只鸡，问有多少种购买方式？
8. 编写程序打印出所有的三位数的“水仙花数”。“水仙花数”是指一个三位数，其各位数字的立方和等于该数本身。例如，153就是一个水仙花数， $1^3+5^3+3^3=153$ 。



习题

9. 编程解决鸡兔同笼问题：已知鸡有2条腿，兔有4条腿，现在鸡兔同笼，已知总共有 n 个头和 m 条腿，求鸡和兔各多少只。
10. 哥德巴赫猜想指出，每个不小于6的偶数都可以表示为两个质数之和。编写一个程序验证这一猜想，对用户输入的任意一个不小于6的偶数，输出对应的两个质数之和。
11. 修改SIR传染模型，加入疫苗接种因素，观察并分析疫苗接种对传播动态的影响。
12. 在SIR模型中加入超级传播者的概念，模拟并分析其对传染病传播速度和范围的影响。（注：超级传播者是指感染了大量人群的感染者）。



The End !